

AZ-204 Azure Developer Associate

Day 2: Azure Storage

Mastering Storage Accounts, Blob, Table, Queue, File Storage, Access Tiers, SAS, and SDK-based developer access.

DAY 2 OF 5

AZ-204 TRAINING



Day 2 Agenda

A structured journey through Azure Storage — from core concepts to hands-on SDK usage.

01

Azure Storage Architecture

Storage accounts, replication, and the service model

02

Storage Services Deep Dive

Blob, Table, Queue, and File Storage

03

Access Tiers & Lifecycle

Hot, Cool, Archive — and automated lifecycle policies

04

Security & SDK Access

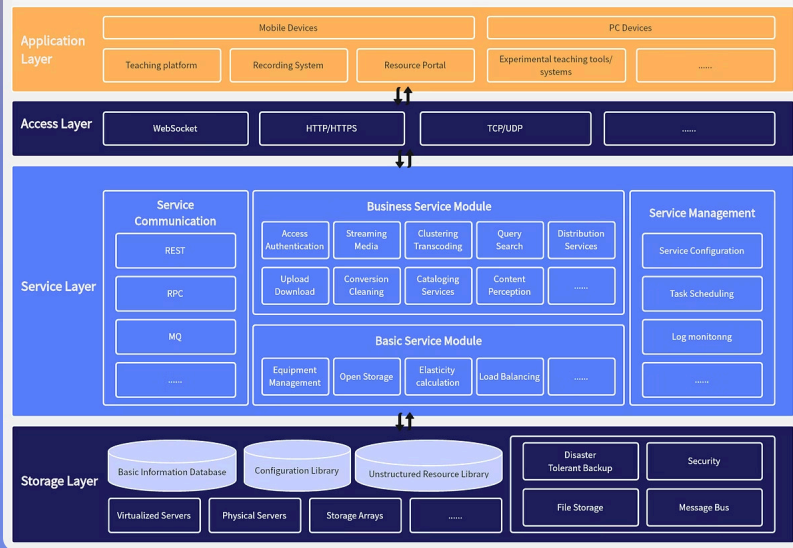
Shared Access Signatures, keys, and developer SDKs

05

Real-World Scenarios & Labs

File upload systems, logging pipelines, async processing

Digital Platform Architecture Diagram

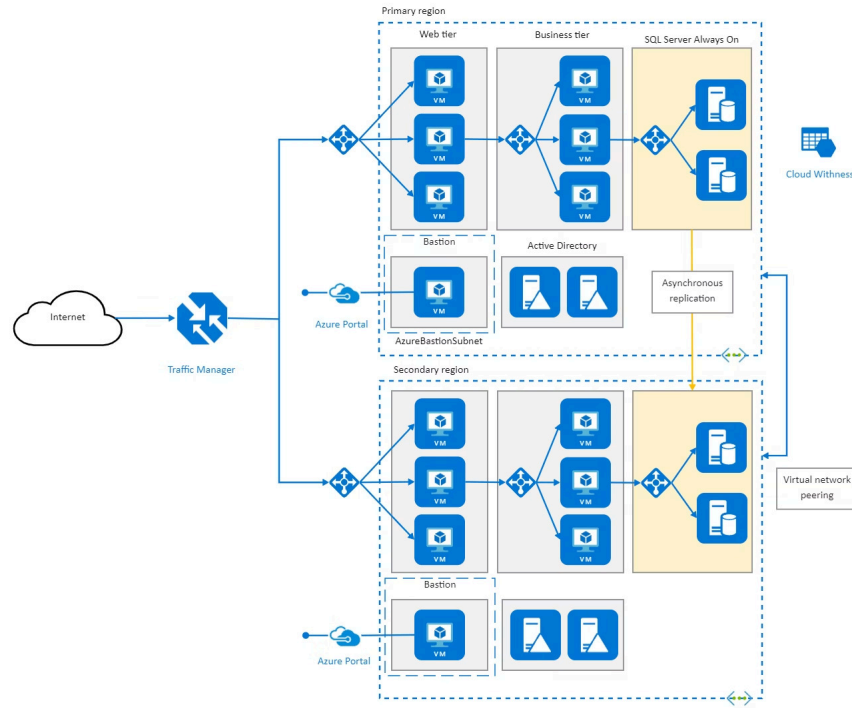


CHAPTER 1

Azure Storage Architecture

Understanding how Azure organizes and replicates data at global scale.

Azure Storage: The Big Picture



Azure Storage is a **managed, globally distributed** cloud storage platform. Every piece of data lives inside a **Storage Account** — the top-level resource that groups services, billing, and access control.

Durability

Automatic replication across nodes and regions

Scalability

Petabyte scale with no infrastructure management

Security

Encryption at rest and in transit by default

Storage Account: The Foundation

A Storage Account is the **namespace and billing boundary** for all Azure Storage services. Every resource — blobs, queues, tables, and files — lives inside one.

Standard (GPv2)

General-purpose v2 — supports all storage services. The default choice for most workloads.

Premium Block Blob

SSD-backed, ultra-low latency for high-throughput blob workloads.

Premium File Share

High-performance Azure Files for enterprise and lift-and-shift scenarios.

Premium Page Blob

Optimized for random read/write workloads like Azure Virtual Machine disks.

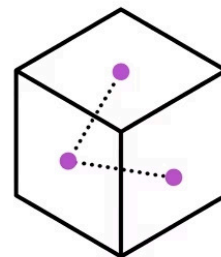
Replication Options

Azure replicates your data automatically. Choose a replication strategy based on your **durability requirements** and **budget**.

- **LRS** — Locally Redundant Storage: 3 copies in one datacenter
- **ZRS** — Zone Redundant Storage: 3 copies across availability zones
- **GRS** — Geo Redundant Storage: 6 copies across two regions
- **GZRS** — Geo-Zone Redundant: ZRS + geo-replication combined

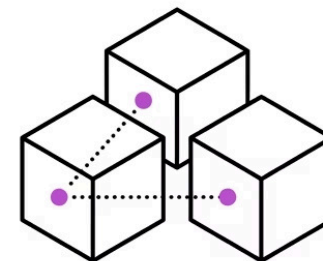
📄 All options provide at least **11 nines (99.999999999%)** data durability.

LRS



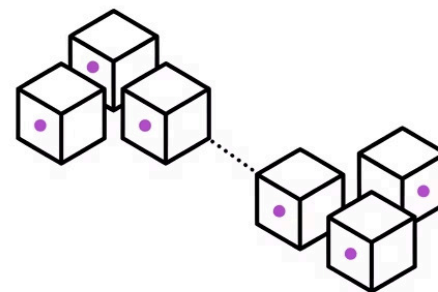
3 copies, 1 datacenter

ZRS



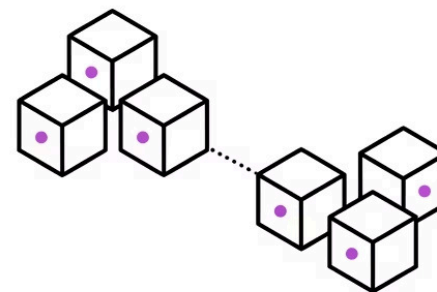
3 copies, 3 zones

GRS



6 copies, 2 regions

GZRS



6 copies, zones + regions

CHAPTER 2

Azure Storage Services

Blob, Table, Queue, and File — four purpose-built services for every developer use case.



Four Storage Services at a Glance



Blob Storage

Unstructured object storage for files, images, videos, backups, and logs. The workhorse of Azure Storage.



Table Storage

NoSQL key-value store for semi-structured data. Ideal for simple lookups, metadata, and audit logs.



Queue Storage

Message queuing service for decoupled, asynchronous communication between application components.



File Storage

Fully managed SMB/NFS file shares. Perfect for lift-and-shift apps that need a network file system.

Table Storage

What it is

A **schema-less NoSQL key-value store** built for fast, cheap lookups on massive datasets. Think of it as a giant dictionary keyed on `PartitionKey + RowKey`.

Developer Use Cases

- User profile metadata
- Audit and activity logs
- IoT telemetry data
- Session state storage

PartitionKey

Groups related rows — drives horizontal scalability

RowKey

Unique identifier within a partition

Timestamp

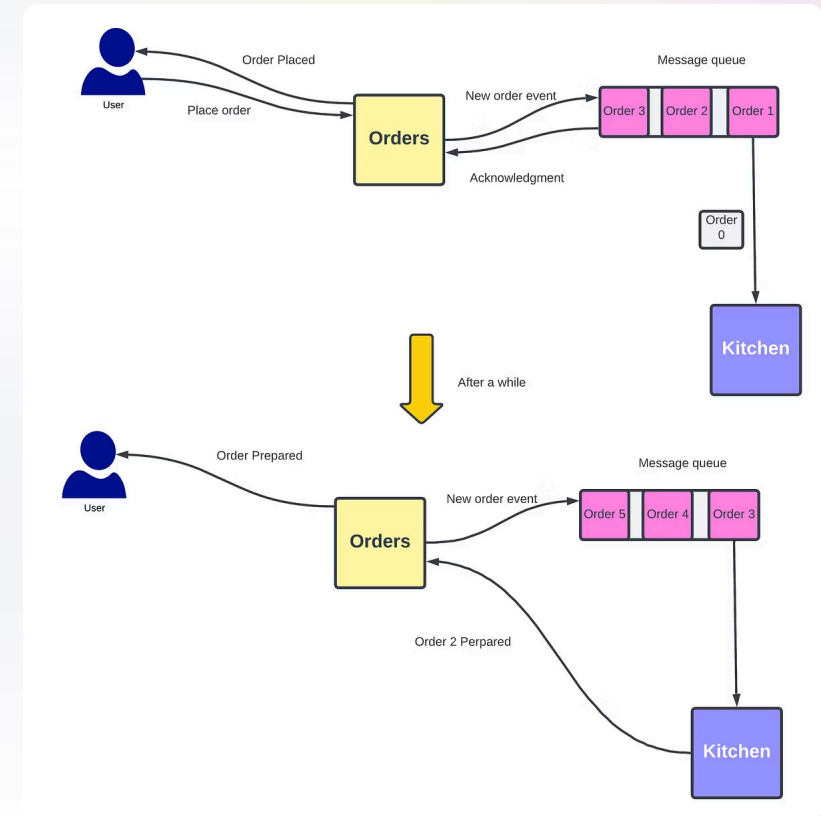
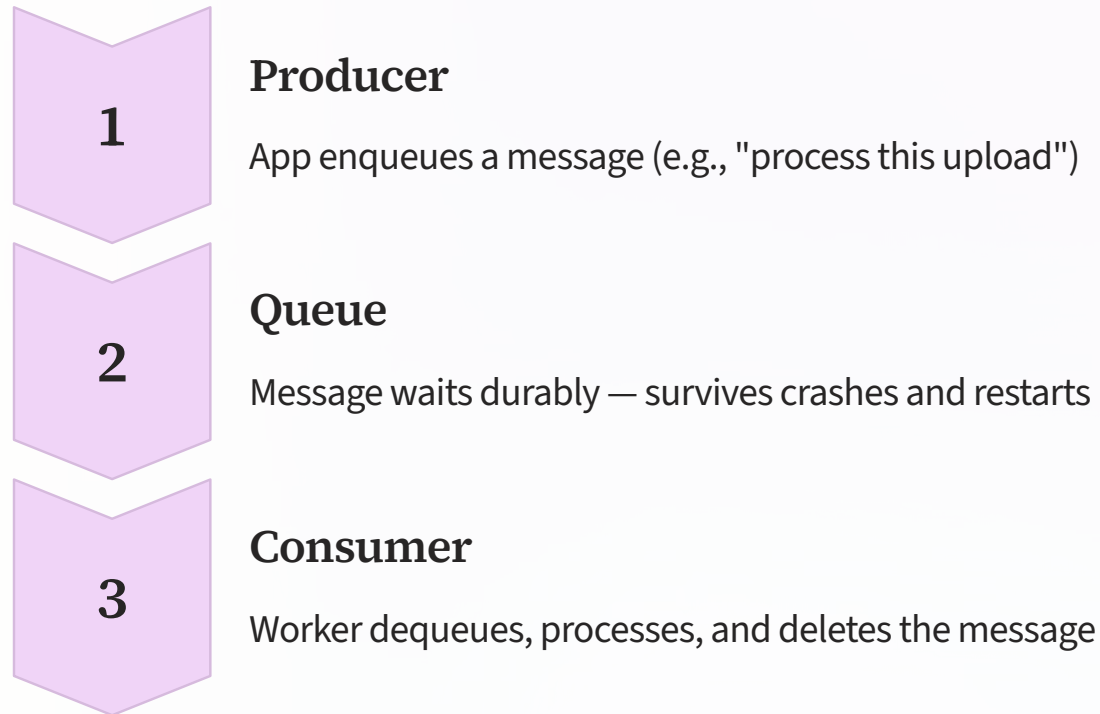
Auto-managed by Azure for optimistic concurrency

Properties

Up to 252 custom typed properties per entity

Queue Storage

Queue Storage enables **reliable async messaging** between decoupled services. Each queue can hold millions of messages up to **64 KB each**, with messages retained for up to **7 days**.



Azure File Storage



Azure Files provides **fully managed cloud file shares** accessible via **SMB** (Windows) and **NFS** (Linux/macOS) protocols — no file server to manage.

Key Developer Scenarios

- **Lift & shift** legacy apps that rely on shared file systems
- Shared config and certificates across multiple VMs
- Replace on-prem file servers with no code changes
- Mount as a drive letter on Windows or a mount point on Linux

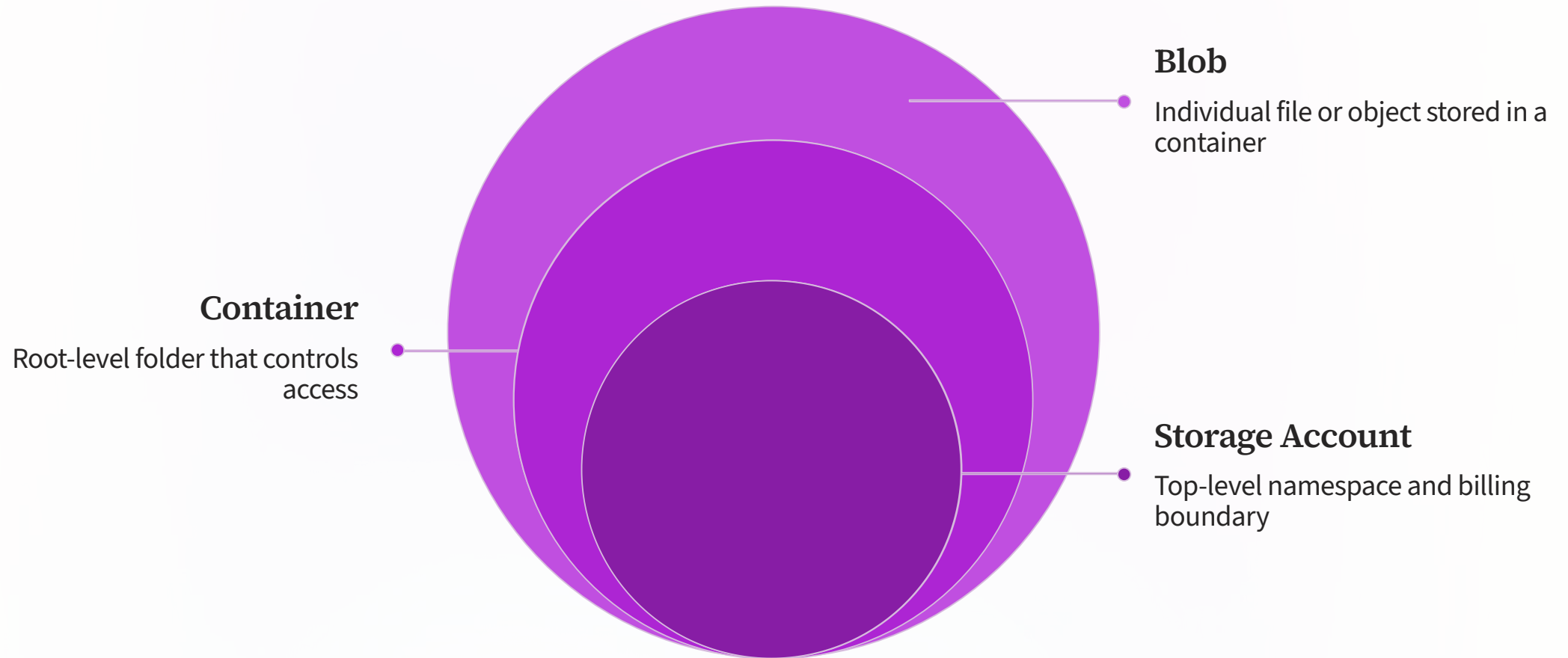


CHAPTER 3

Blob Storage Deep Dive

Containers, blob types, access patterns, and lifecycle management — everything a developer needs.

Blob Storage: Structure



Every blob lives inside a **container**, which lives inside a **Storage Account**. Containers are analogous to root-level folders and define the base access policy for all blobs within them.

Three Blob Types

Block Blob

Optimized for **sequential read/write**.

Composed of up to 50,000 blocks.

Ideal for files, images, videos, documents, and backups.

Most commonly used blob type.

Append Blob

Optimized for **append-only operations**. Each write adds to the end of the blob. Perfect for log files, audit trails, and streaming telemetry.

Page Blob

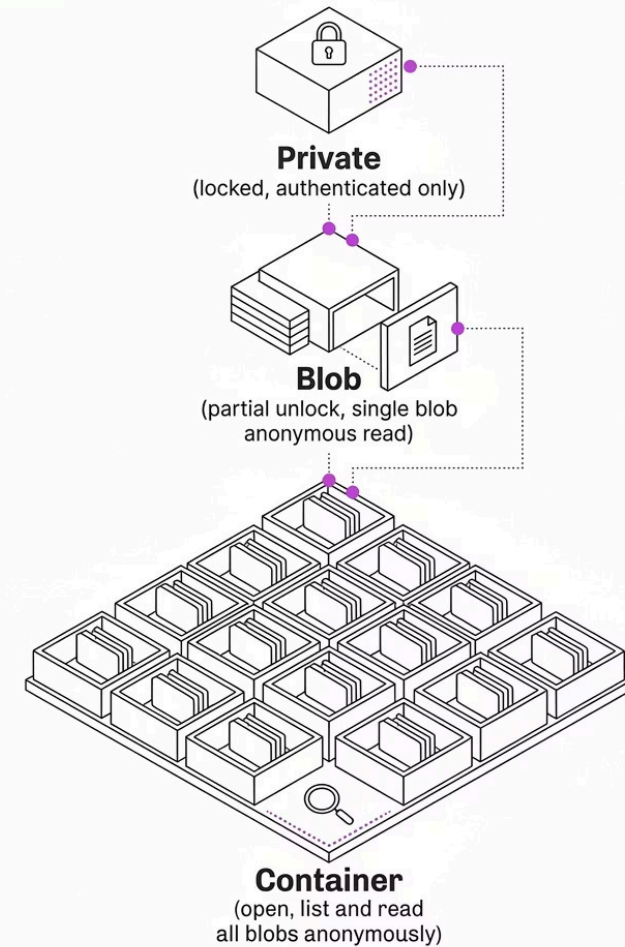
Optimized for **random read/write** access. The underlying storage for **Azure VM disks (VHDs)**. Supports up to 8 TB.

Container Access Levels

Containers have three access levels that control **anonymous (unauthenticated) access** to blobs. Authenticated access is always available regardless of this setting.

- **Private** — No anonymous access. All requests must be authenticated.
- **Blob** — Anonymous read access to individual blobs only.
- **Container** — Anonymous read access to blobs *and* the ability to list all blobs in the container.

📌 **Best practice:** Default to Private. Use SAS tokens for controlled, time-limited sharing.



Blob Access Tiers

Azure Blob Storage offers **three access tiers** to balance storage cost against retrieval frequency. Tiers can be set per blob or as an account default.



Hot

Highest storage cost, **lowest access cost**. For data accessed frequently — active application assets, current datasets.



Cool

Lower storage cost, **moderate access cost**. For infrequently accessed data stored ≥ 30 days — backups, older media files.



Archive

Lowest storage cost, **highest retrieval cost + latency**. For rarely accessed data stored ≥ 180 days — compliance archives, long-term backups.

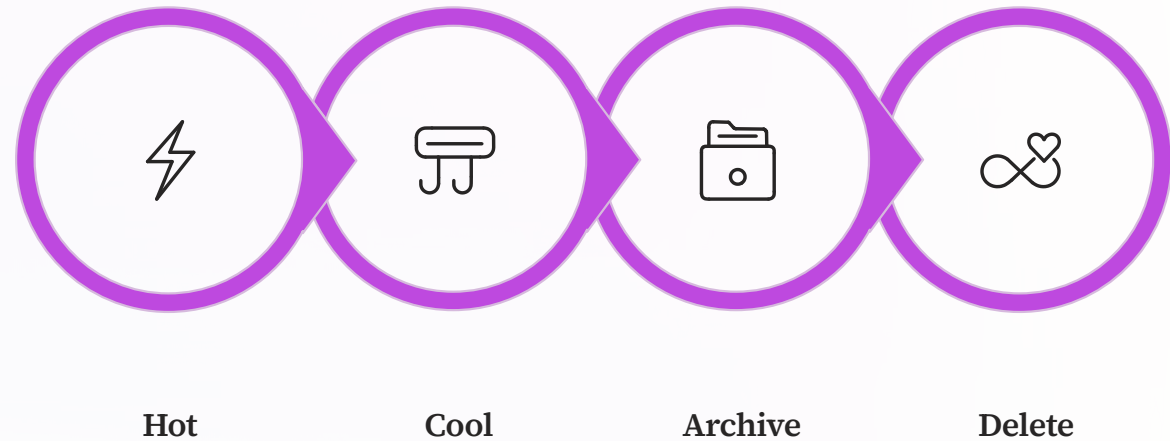
Blob Lifecycle Management

Lifecycle policies let you **automatically transition or delete blobs** based on age and last-access time — eliminating manual cleanup scripts.

Policy Actions

- Transition blobs from Hot → Cool after 30 days
- Transition Cool → Archive after 90 days
- Delete blobs after 365 days
- Apply rules by prefix, container, or blob index tag

 Policies are evaluated daily and applied asynchronously.



Real-World Blob Storage Patterns



File Upload System

User uploads → block blob → process asynchronously. Use SAS URLs for direct browser-to-blob upload, bypassing your API server entirely.

Application Logging

Append blobs accumulate log lines without overwriting. Cheap, durable, and queryable via Azure Monitor or exported to analytics tools.



Static Content & CDN

Host images, CSS, JS, and videos in blob containers. Pair with Azure CDN for global low-latency delivery at scale.

CHAPTER 4

Security & Access Control

Shared Access Signatures, account keys, and Azure AD — controlling who can touch your data.



Access Control Options

1

Account Key

Full root access. Two keys per account for rotation. **Never expose in code** — use Key Vault or environment variables.

2

Shared Access Signature (SAS)

Time-limited, permission-scoped tokens. The preferred approach for **delegating access** to clients or external systems.

3

Azure AD + RBAC

Identity-based access using Managed Identities or service principals. The most secure option for **service-to-service** access.

4

Anonymous Access

Container-level toggle for public blobs. Use sparingly — only for truly public, read-only content like public website assets.

Shared Access Signatures (SAS)

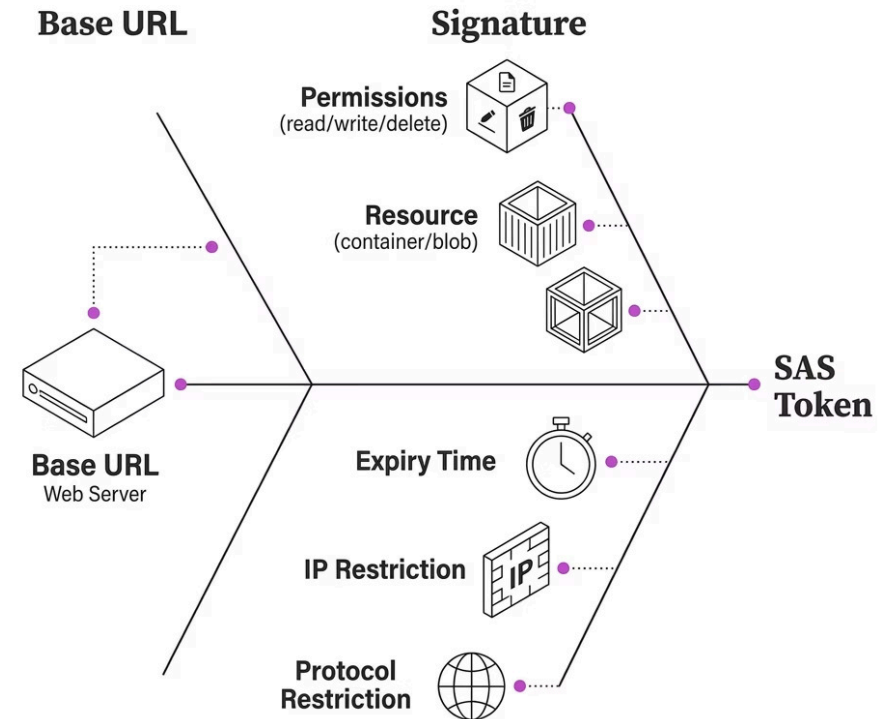
A SAS is a **signed URI** that grants scoped, time-limited access to storage resources — without sharing your account key.

Three SAS Types

- **Account SAS** — Access to one or more storage services
- **Service SAS** — Access to a specific service (e.g., Blob only)
- **User Delegation SAS** — Backed by Azure AD credentials — most secure

What You Can Control

- Permissions: read, write, delete, list, create
- Expiry: start and end time
- IP range restriction
- Allowed protocols (HTTPS only)



SAS: Developer Best Practices

→ Use the shortest viable expiry

SAS tokens for client uploads should expire in minutes, not hours. Limit blast radius if a token is leaked.

→ Prefer User Delegation SAS

Backed by Azure AD — no account key involved. Revocable by revoking the identity's role assignment.

→ Use Stored Access Policies

Define permissions server-side and reference them in SAS tokens. Revoke access by modifying the policy — no need to regenerate tokens.

CHAPTER 5

SDK-Based Developer Access

Interacting with Azure Storage programmatically using the official .NET, Python, and JavaScript SDKs.

Azure Storage SDK Overview

The Azure Storage SDKs provide **idiomatic, type-safe clients** for all storage services. Available for .NET, Python, Java, JavaScript/TypeScript, and Go.

BlobServiceClient

Top-level client. Manages containers — create, list, delete.
Entry point for all blob operations.

BlobContainerClient

Operates on a specific container. List blobs, set access policies, generate SAS tokens.

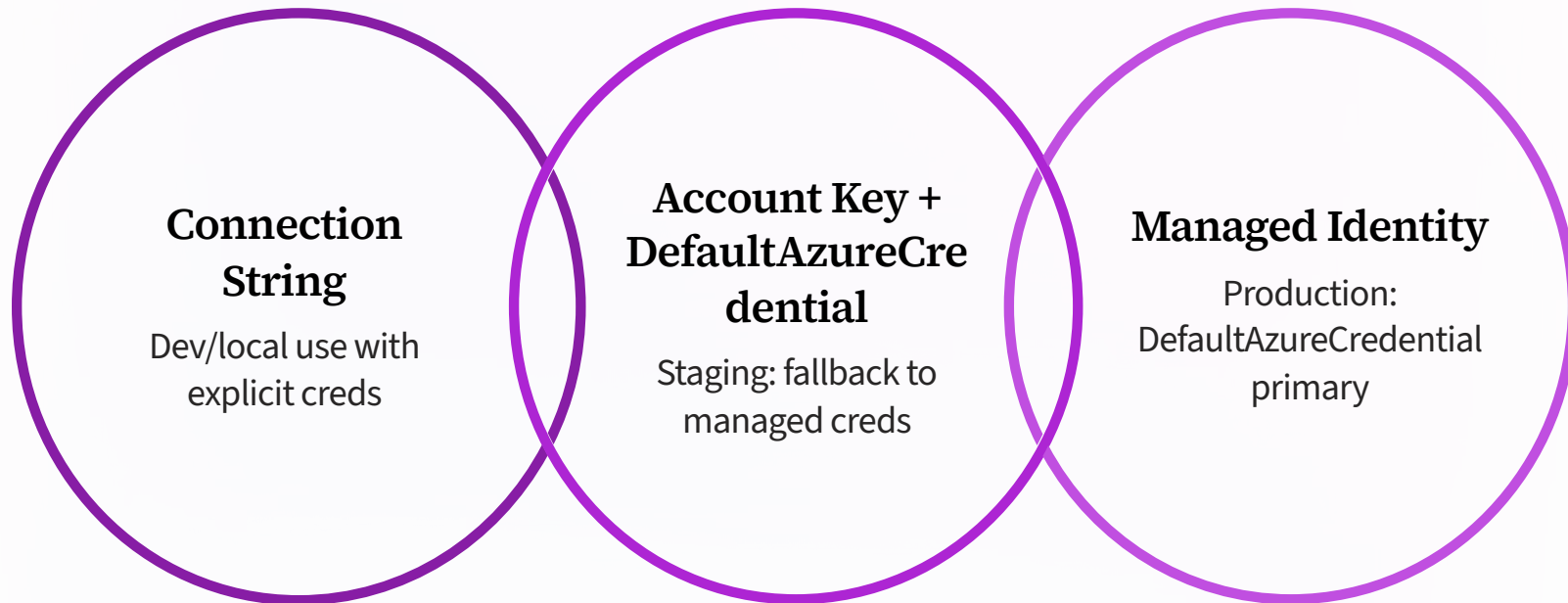
BlobClient

Upload, download, delete, and set metadata on a single blob.
Supports streaming for large files.

BlockBlobClient

Fine-grained control over block uploads. Stage blocks individually, then commit — ideal for large or resumable uploads.

SDK Authentication Patterns



Use `DefaultAzureCredential` for a **seamless local-to-production auth strategy** — it tries multiple credential sources automatically, so you write auth code once and it works everywhere from your laptop to Azure App Service.

Common SDK Operations: Blob

Upload a Blob

```
var client = new BlobClient(  
    connStr, "uploads", "photo.jpg");  
await client.UploadAsync(  
    stream, overwrite: true);
```

Generate a SAS URL

```
var sasUri = client.GenerateSasUri(  
    BlobSasPermissions.Read,  
    DateTimeOffset.UtcNow  
        .AddMinutes(15));
```

Download a Blob

```
var response = await client  
    .DownloadStreamingAsync();  
await response.Value.Content  
    .CopyToAsync(localStream);
```

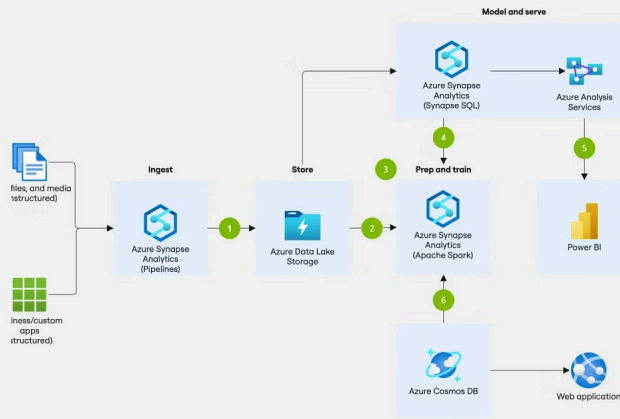
Set Blob Metadata

```
await client.SetMetadataAsync(  
    new Dictionary<string, string> {  
        { "uploadedBy", "userId123" },  
        { "contentType", "photo" }  
    });
```

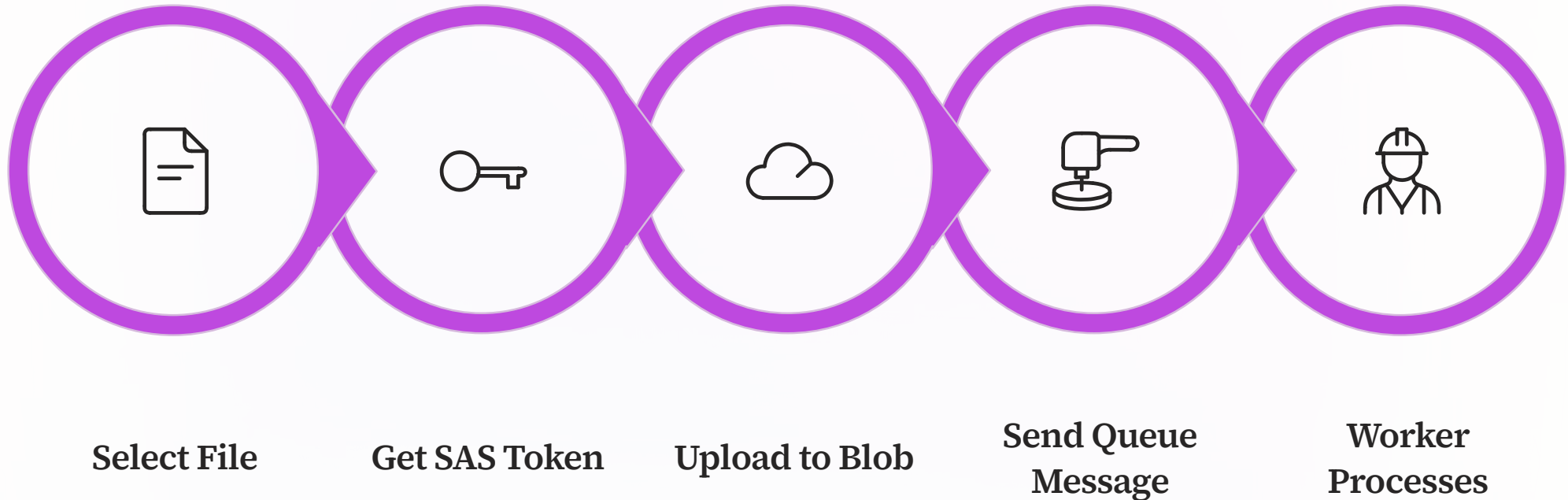
CHAPTER 6

Real-World Scenarios

Putting it all together — practical patterns that appear in production Azure applications.



Scenario: File Upload Web App



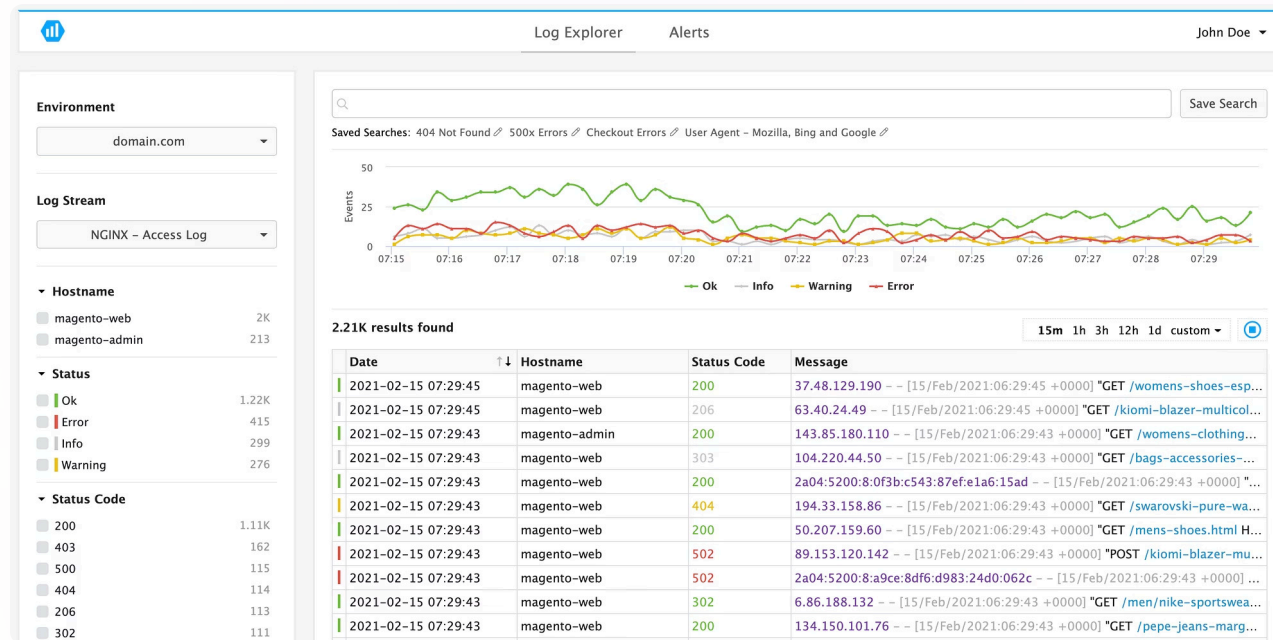
This pattern offloads bandwidth from your API server — the browser uploads **directly to Blob Storage** using a short-lived SAS URL. A queue message triggers async post-processing (virus scan, thumbnail generation, etc.).

Scenario: Centralized Application Logging

Architecture

- Multiple app instances write to **append blobs** — one blob per app, per day
- Naming convention: logs/app-2024-01-15.log
- Lifecycle policy archives blobs after 30 days, deletes after 365
- Export to Azure Monitor or Data Lake for querying

Append blobs are **atomic** per append operation — safe for concurrent writers.



Scenario: Async Order Processing

Queue Storage decouples your **web frontend** from your **order processing backend** — a classic pattern for resilient, scalable systems.



Resilience

Queue persists messages even if the worker is down — no lost orders

Scalability

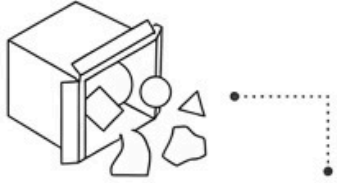
Scale workers independently based on queue depth

Retry Safety

Visibility timeout prevents duplicate processing — messages reappear if not deleted

Choosing the Right Storage Service

Use this quick reference when designing your storage architecture.



Blob Storage for unstructured files and objects.

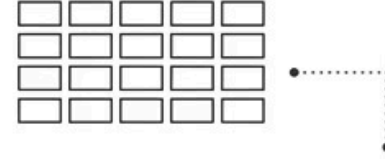
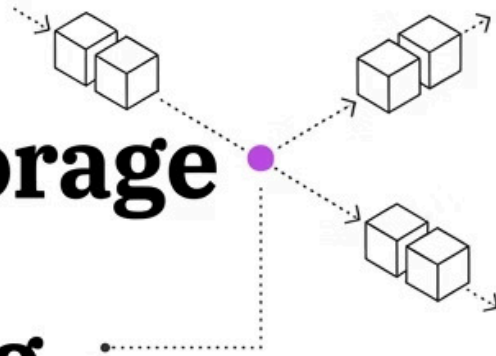
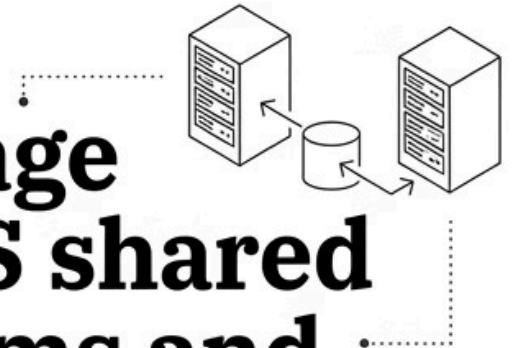


Table Storage for NoSQL key-value data and metadata.

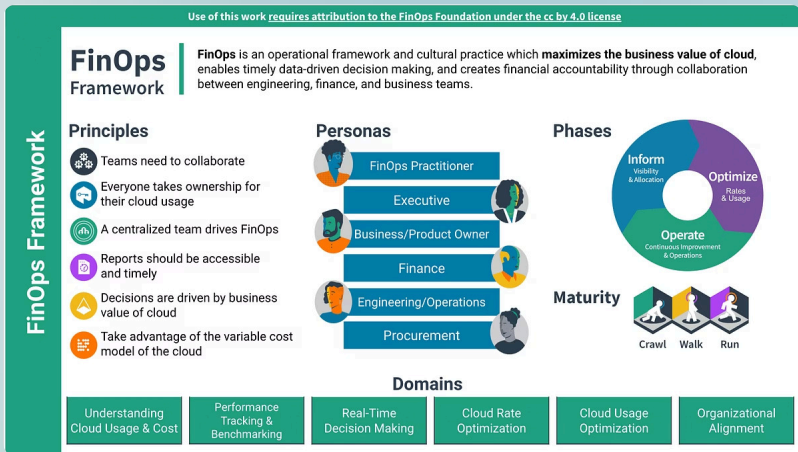
Queue Storage for async messaging between services.



File Storage SMB/NFS shared file systems and legacy lift-and-shift.



Storage Cost Optimization Tips



Use lifecycle policies aggressively

Automatically move aging data to Cool and Archive. Most apps keep data hotter than necessary — set it and forget it.

Right-size your replication

LRS is fine for dev/test and non-critical data. Reserve GRS/GZRS for data that must survive a regional outage.

Delete uncommitted blocks

Failed or partial uploads leave staged blocks that incur charges. Enable auto-deletion of uncommitted blocks after 7 days.

Key Concepts: Quick Reference

4

Storage Services

Blob, Table, Queue, File

3

Blob Types

Block, Append, Page

3

Access Tiers

Hot, Cool, Archive

3

SAS Token Types

Account, Service, User
Delegation

11x

Nines of Durability

99.999999999% for all tiers

Day 2 Labs

Today's hands-on labs are designed to reinforce the concepts covered in this session. Your instructor will provide detailed lab guides.

1

Create and Configure a Storage Account

2

Blob Storage Operations using SDK

3

Secure Access using SAS Tokens

4

Build a File Upload Web App

Day 2 Summary: What We Covered

Storage Architecture

Accounts, service types, replication strategies, and the Azure Storage hierarchy

Blob Storage

Containers, blob types (block/append/page), access tiers, and lifecycle policies

Table, Queue & File

NoSQL lookups, async messaging, and SMB/NFS file sharing for developers

Security & SAS

Account keys, SAS token types, scoping permissions, and best practices

SDK Access

Client hierarchy, DefaultAzureCredential, and common blob SDK operations